

Klasa Applet

Zbigniew Marszalek

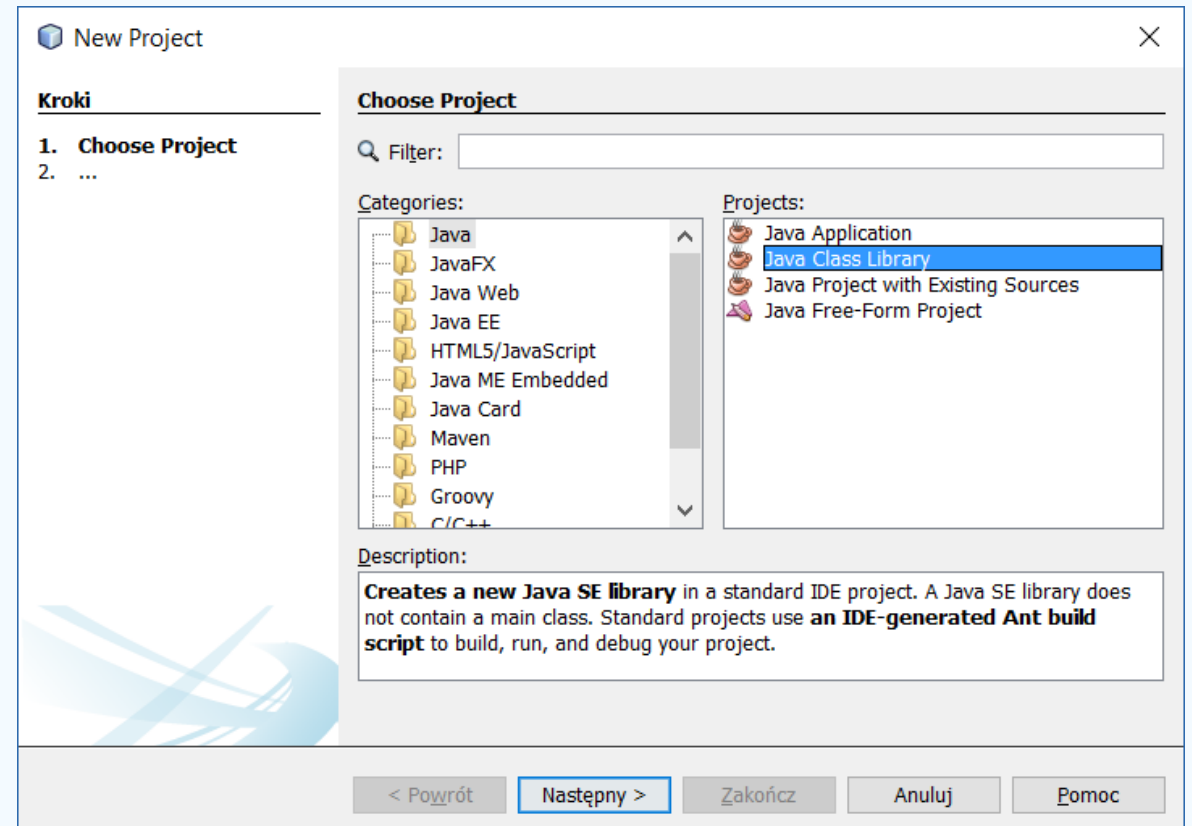
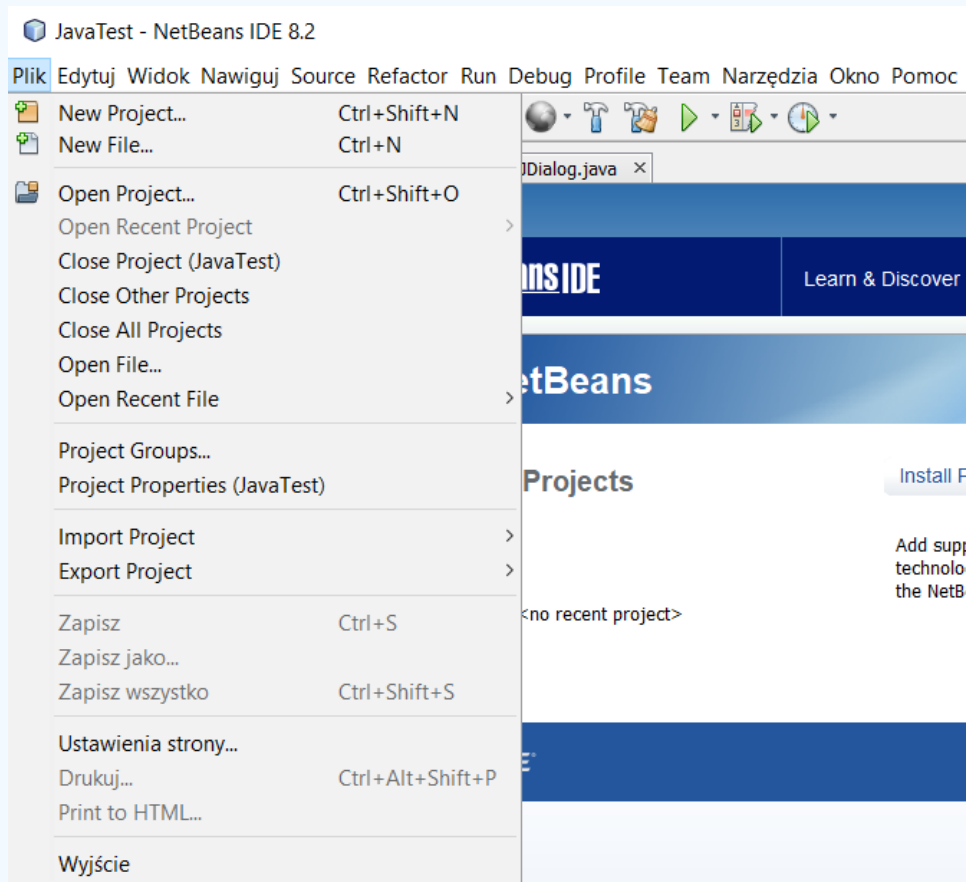
Klasa Applet

Klasa Applet jest częścią składową pakietu `java.applet`. Klasa ta zawiera metody umożliwiające konstrukcje apletów i ich wykonywanie. Pakiet `java.applet` definiuje trzy interfejsy: `AppletContext`, `AudioClip` i `AppletStub`.

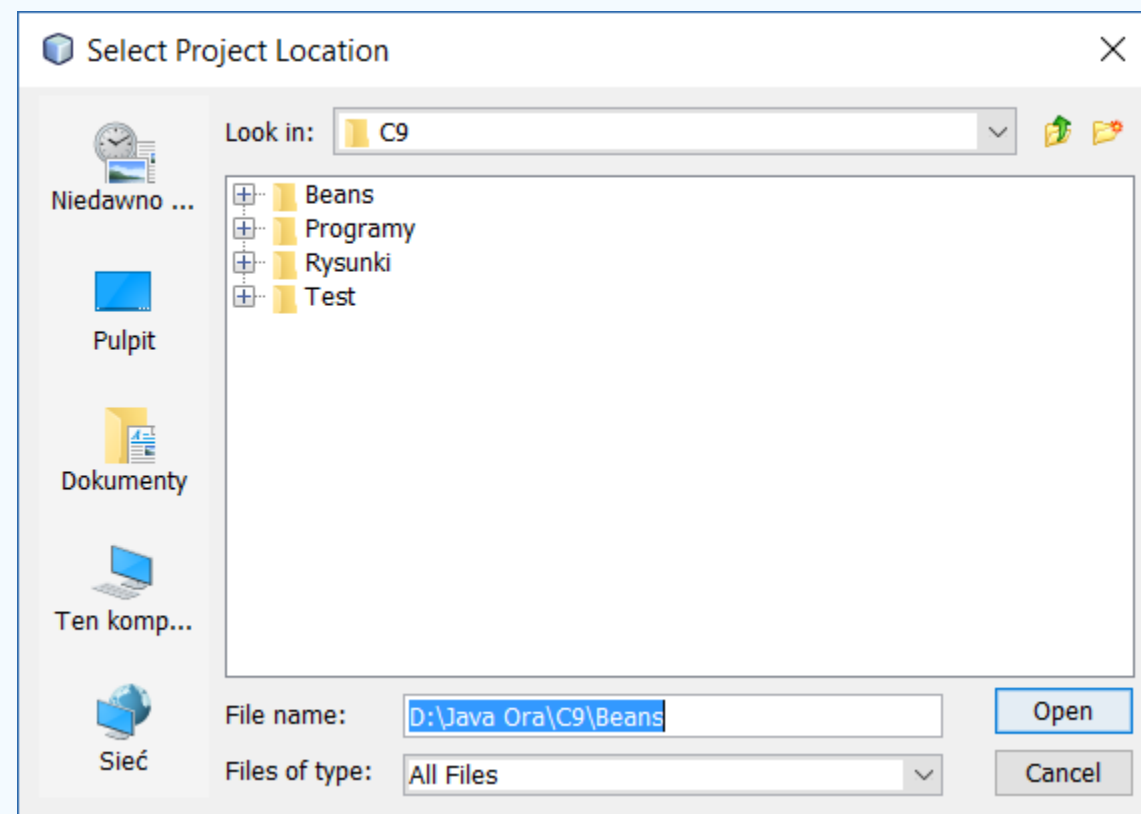
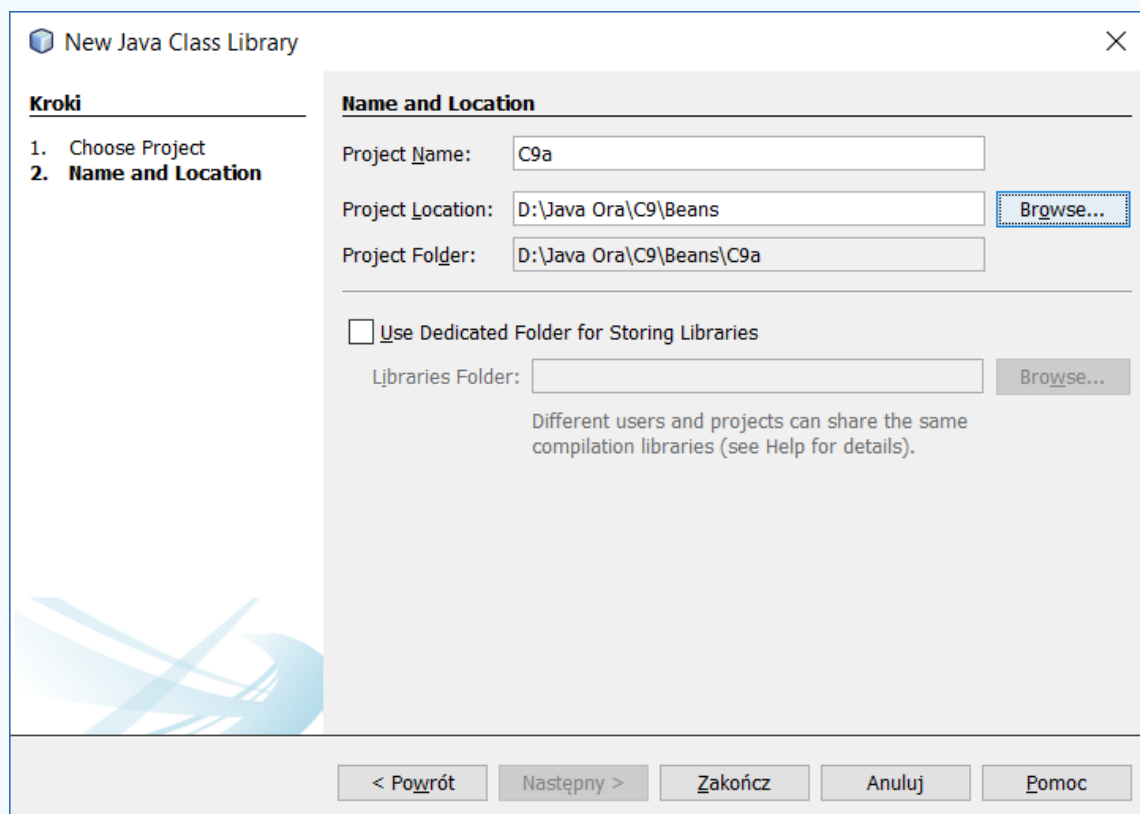
Wszystkie Aplety są podklasami klasy Applet. Oznacza to, że wszystkie aplety muszą importować pakiet `java.applet` i `java.awt` (ang. Abstract Window Toolkit). Ponieważ wszystkie aplety działają w swoich oknach, niezbędne jest importowanie mechanizmu obsługującego te okna.

Aplety nie są uruchamiane z poziomu konsoli przez interpreter czasu wykonywania środowiska Javy, ale przez przeglądarkę internetową lub przez przeglądarkę apletów. Ilustracje prezentowane w tym wykładzie zostały przygotowane w przeglądarce NetBeans IDE lub w przeglądarce apletów `appletviewer` i udostępnionej w ramach JDK.

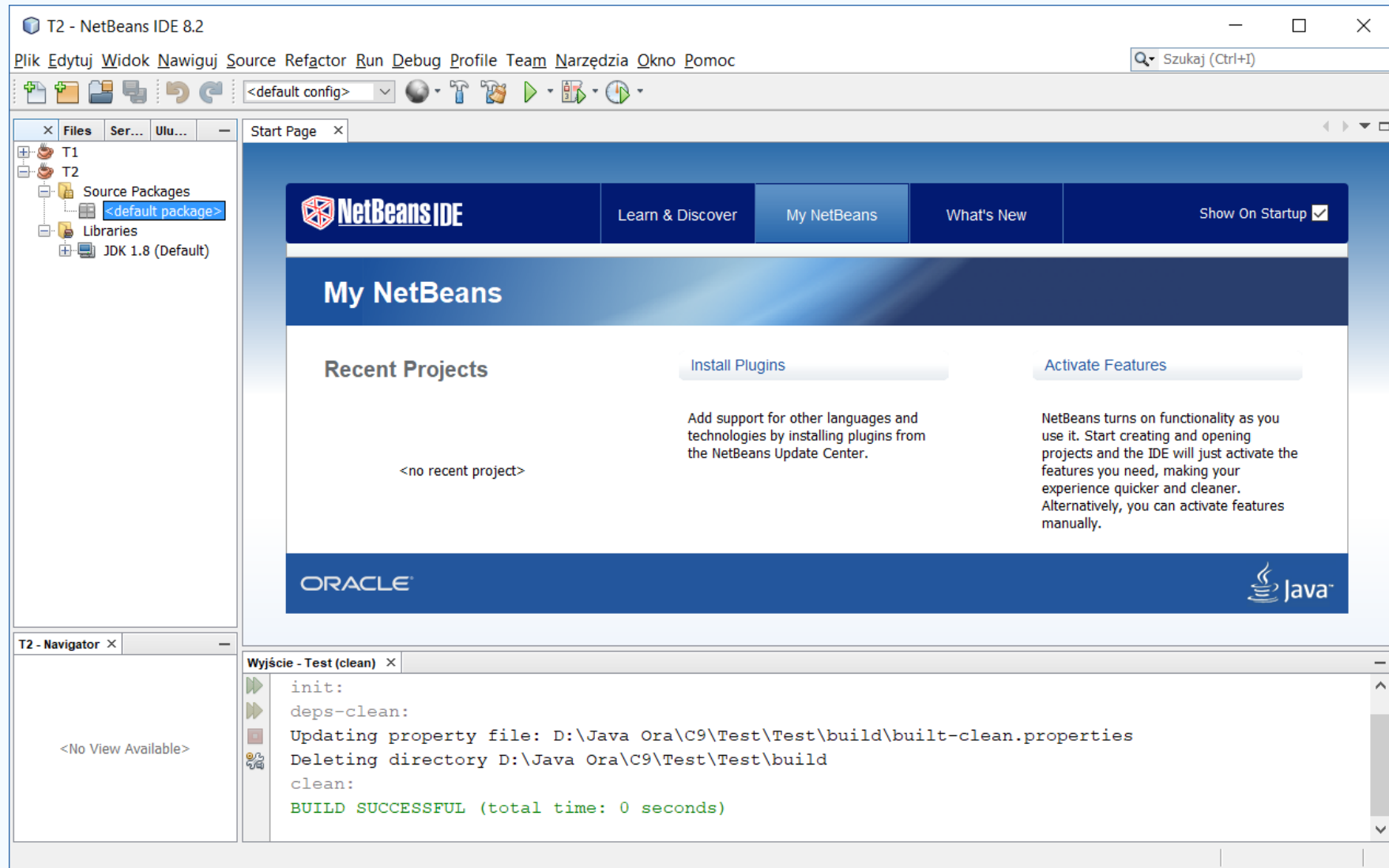
Budujemy nowy projekt w programie Net Beans IDE klikają w zakładce Plik pozycję New Project. (Ctrl+Shift+N). Można to zrobić prościej, kliknąć w menu Net Beans w ikonę Nowy Projekt. W ukazującym się oknie wybieramy pozycję Java oraz Java Class Library i klikamy w przycisk Następny>.



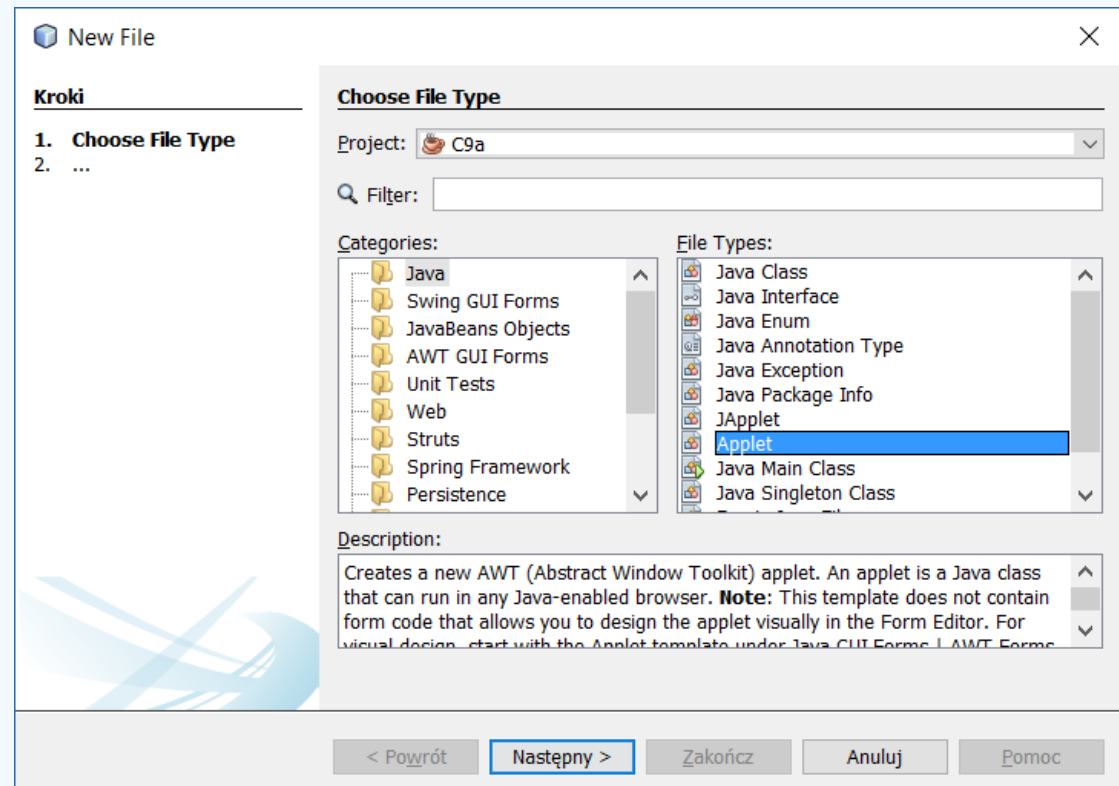
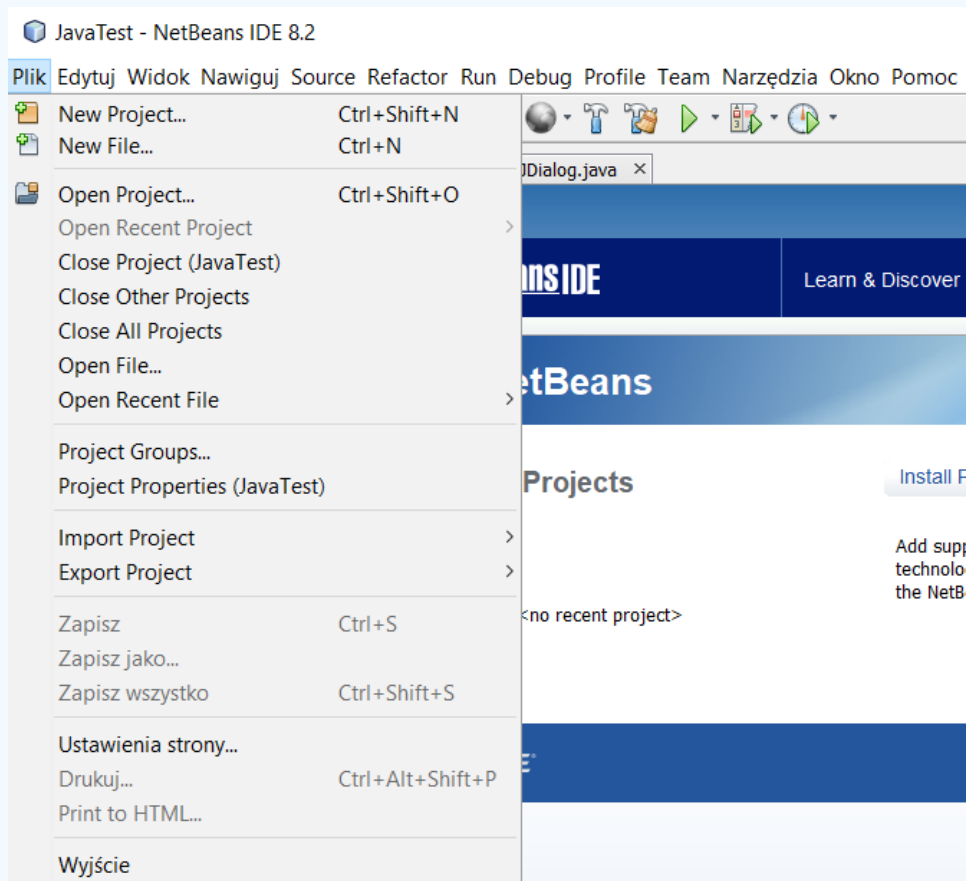
W pojawiającym się oknie wpisujemy nazwę projektu i za pomocą przycisku Browse wybieramy lokalizację naszego projektu. Następnie klikamy w zakładkę Zakończ.



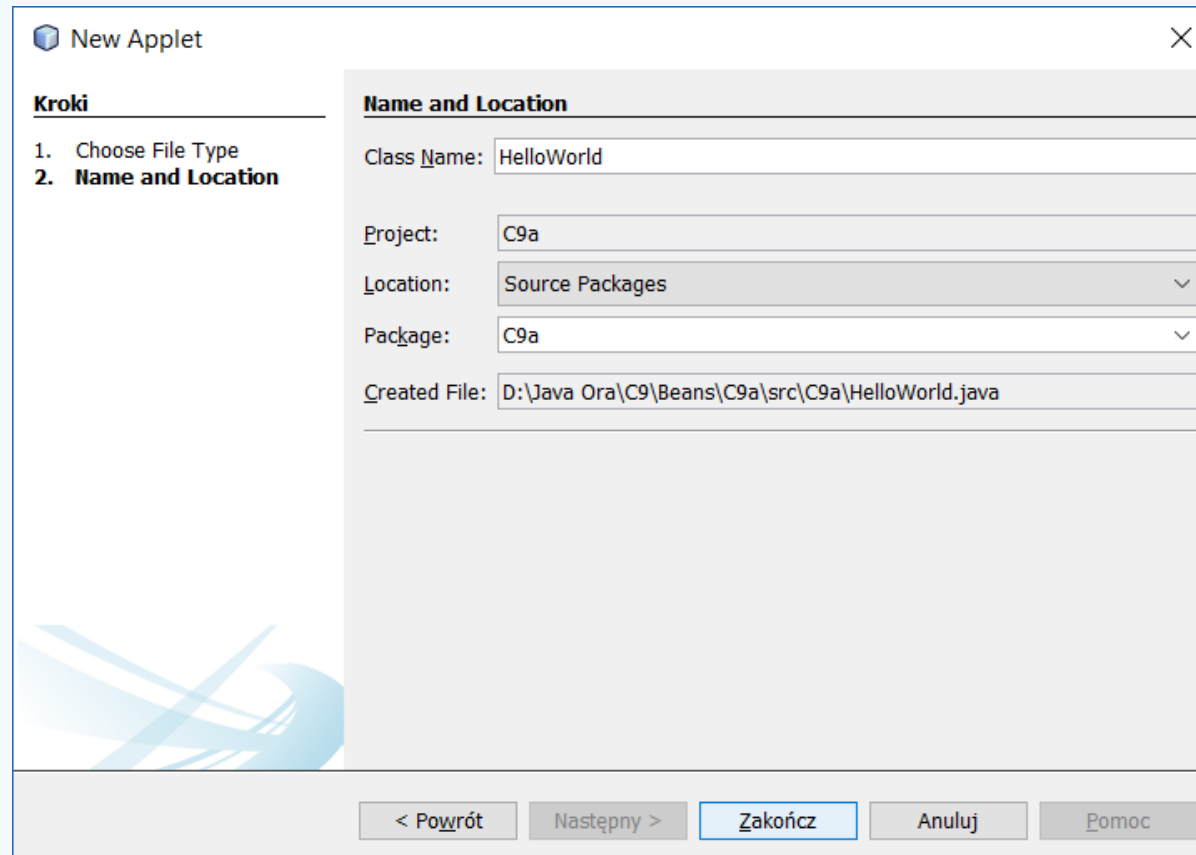
Okno projektu aplikacji w języku Java



W ten sposób otrzymujemy szablon pustego projektu w języku Java. Do otrzymanego projektu dodajmy kolejny element wybierając w zakładce Plik pozycję New File (Ctrl+N) lub klikając w ikonę reprezentującą nowy plik. W ukazującym się oknie wybieramy pozycję Java i Applet. W wyświetlonym oknie klikamy w przycisk Następny>.



W pojawiającym się oknie wpisujemy nazwę apletu i pakietu. Następnie klikamy w zakładkę Zakończ. Po utworzeniu okna klasy apletu pisujemy kod programu apletu.



New Applet

Kroki

1. Choose File Type
2. **Name and Location**

Name and Location

Class Name: HelloWorld

Project: C9a

Location: Source Packages

Package: C9a

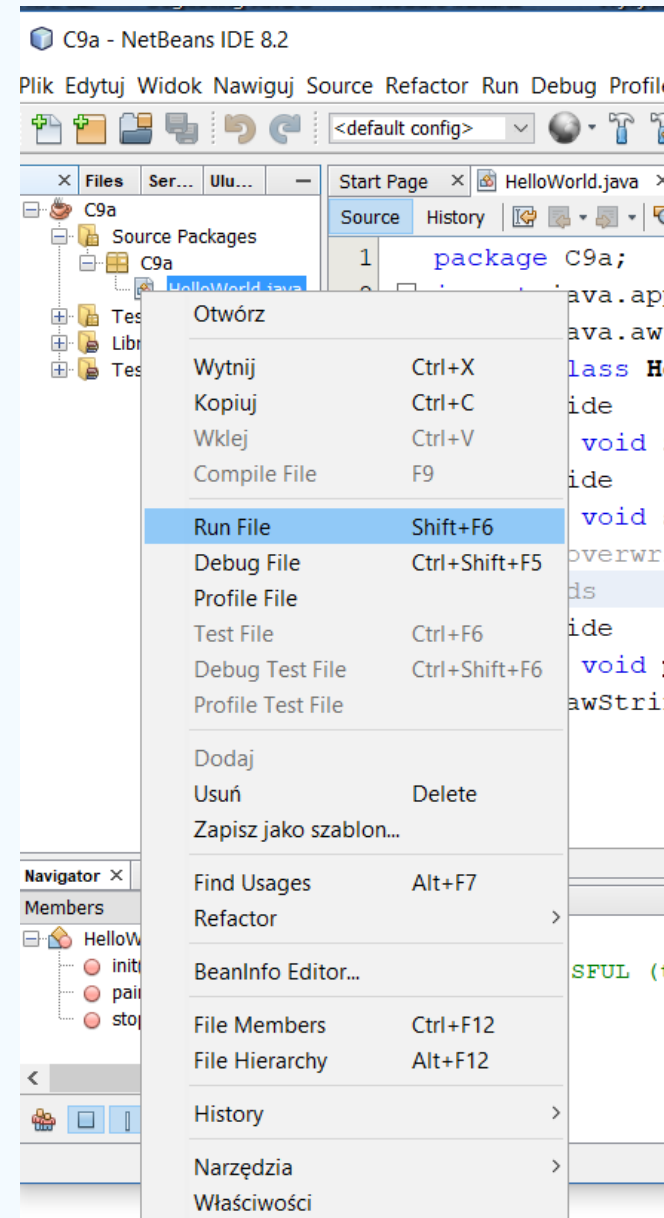
Created File: D:\Java Ora\C9\Beans\C9a\src\C9a\HelloWorld.java

< Powrót Następny > **Zakończ** Anuluj Pomoc

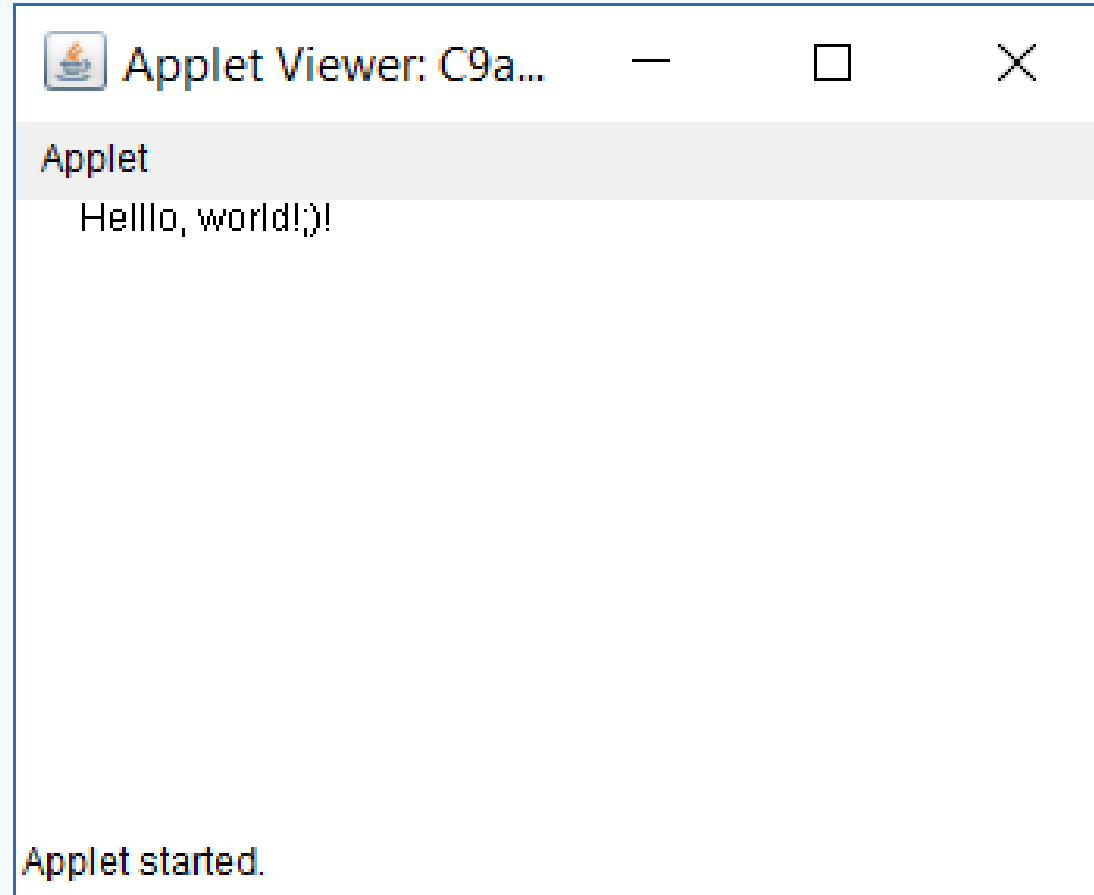
```
package C9a;
import java.applet.Applet;
import java.awt.*;
public class HelloWorld extends Applet {
    @Override
    public void init() {}
    @Override
    public void stop() {}
    // TODO overwrite start(), stop() and destroy()
    // methods
    @Override
    public void paint(Graphics g) {
        g.drawString("Hello, world!;)", 20, 10);
    }
}
```

Uruchamianie i wyświetlenie apletu

Aplety nie są uruchamiane z poziomu konsoli przez interpreter czasu wykonywania środowiska Javy dlatego utworzony plik należy uruchomić w przeglądarce apletów. Można tego dokonać klikając prawym klawiszem myszy w ikonę utworzonego pliku w nawigatorze projektów i wybierając pozycję w wyświetlanym oknie Run File (Shift+F6).

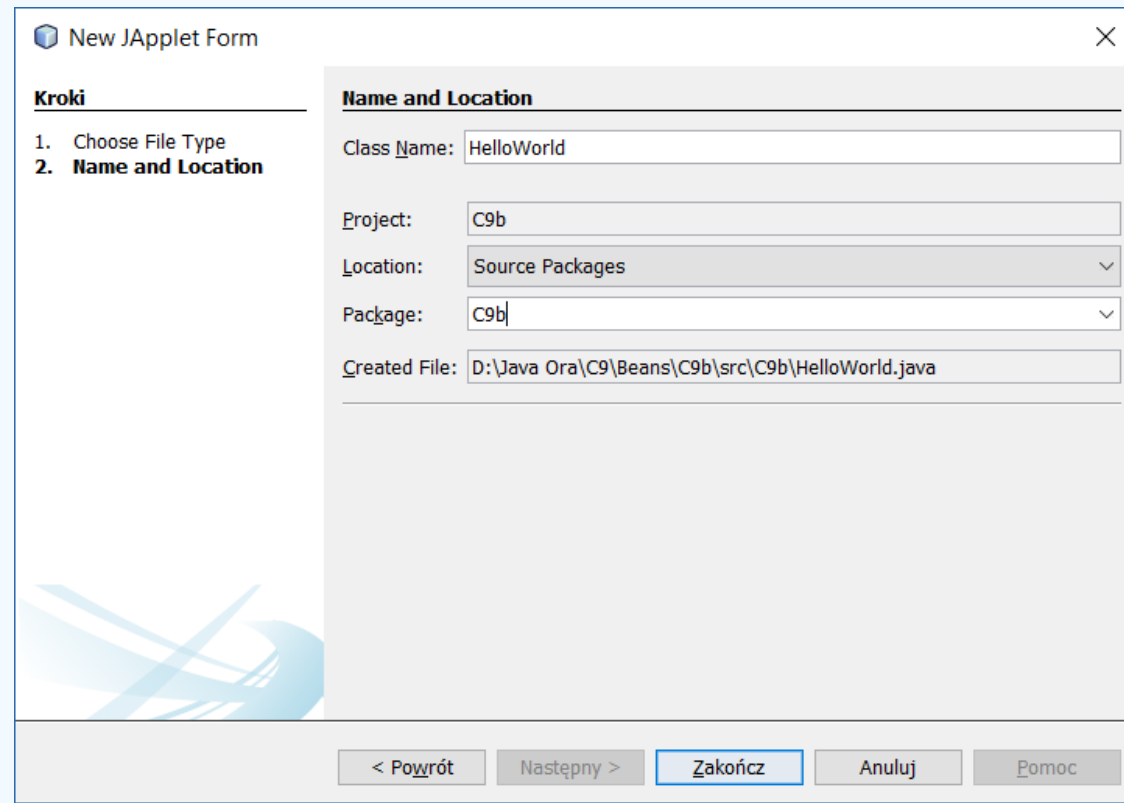
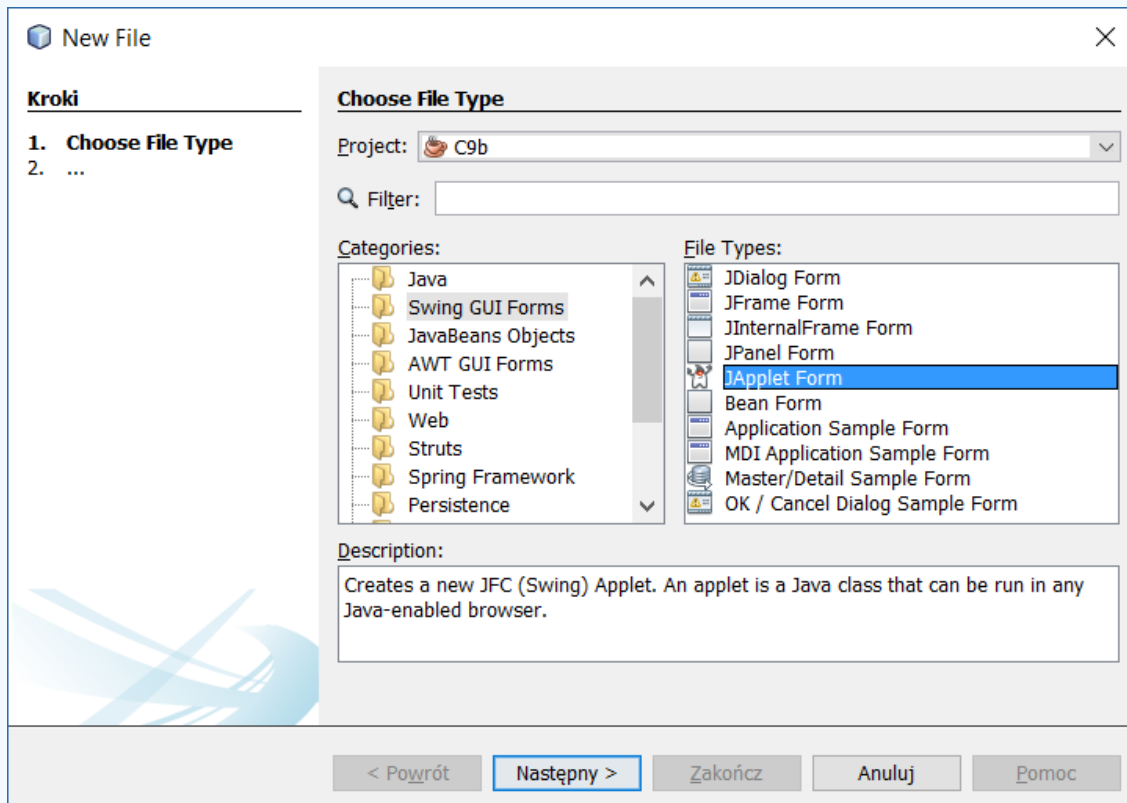


Wyświetlenie apletu w przeglądarce NetBeans IDE

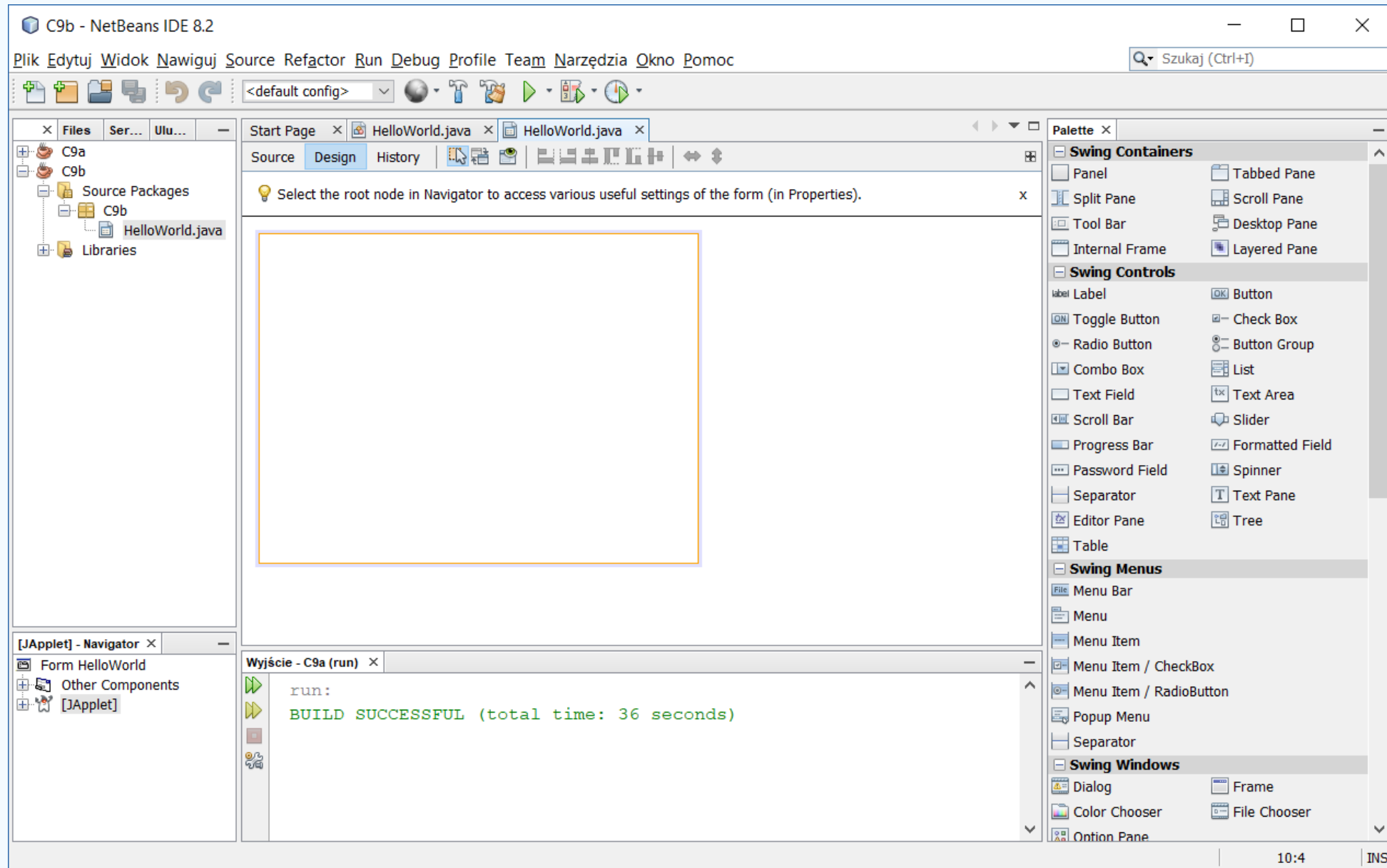


Kreator apletów w NetBeans IDE

W celu wygenerowania projektu umożliwiającego tworzenie apletów za pomocą kreatora budujemy najpierw pusty projekt biblioteki klas tak jak zostało to poprzednio pokazane. Następnie dodajemy do projektu formularz apletów wybierając w ukazującym się oknie Swing GUI Forms i JApplet Form. Klikamy w przycisk Następny >. W ukazującym się oknie wpisujemy nazwę klasy i nazwę projektu. Klikamy w przycisk Zakończ.



Formularz apletu



```
package C9b;

public class HelloWorld
    extends javax.swing.JApplet {
    @Override
    public void init() {
        /* Set the Nimbus look and feel */
        //<editor-fold defaultstate="collapsed" desc="
        //      Look and feel setting code (optional) ">
        /* If Nimbus (introduced in Java SE 6) is not
        *available, stay with the default look and feel.
        * For details see http://download.oracle.com/
        * javase/tutorial/uiswing/lookandfeel/plaf.html
        */
```

```
try {  
    for (javax.swing.UIManager.LookAndFeelInfo info:  
        javax.swing.UIManager.getInstalledLookAndFeels())  
    {  
        if ("Nimbus".equals(info.getName())) {  
            javax.swing.UIManager.setLookAndFeel  
                (info.getClassName());  
            break;  
        }  
    }  
} catch (ClassNotFoundException ex) {  
    java.util.logging.Logger.getLogger  
        (HelloWorld.class.getName()).log  
        (java.util.logging.Level.SEVERE, null, ex);  
}
```

```
} catch (InstantiationException ex) {  
    java.util.logging.Logger.getLogger  
        (HelloWorld.class.getName()).log  
(java.util.logging.Level.SEVERE, null, ex);  
} catch (IllegalAccessException ex) {  
    java.util.logging.Logger.getLogger  
        (HelloWorld.class.getName()).log  
(java.util.logging.Level.SEVERE, null, ex);  
} catch (javax.swing.  
UnsupportedLookAndFeelException ex) {  
    java.util.logging.Logger.getLogger  
        (HelloWorld.class.getName()).log  
(java.util.logging.Level.SEVERE, null, ex);  
}
```

```
//</editor-fold>
/* Create and display the applet */
try {
    java.awt.EventQueue.invokeLaterAndWait
                                (new Runnable() {
        public void run() {
            initComponents();
        }
    });
} catch (Exception ex) {
    ex.printStackTrace();
}

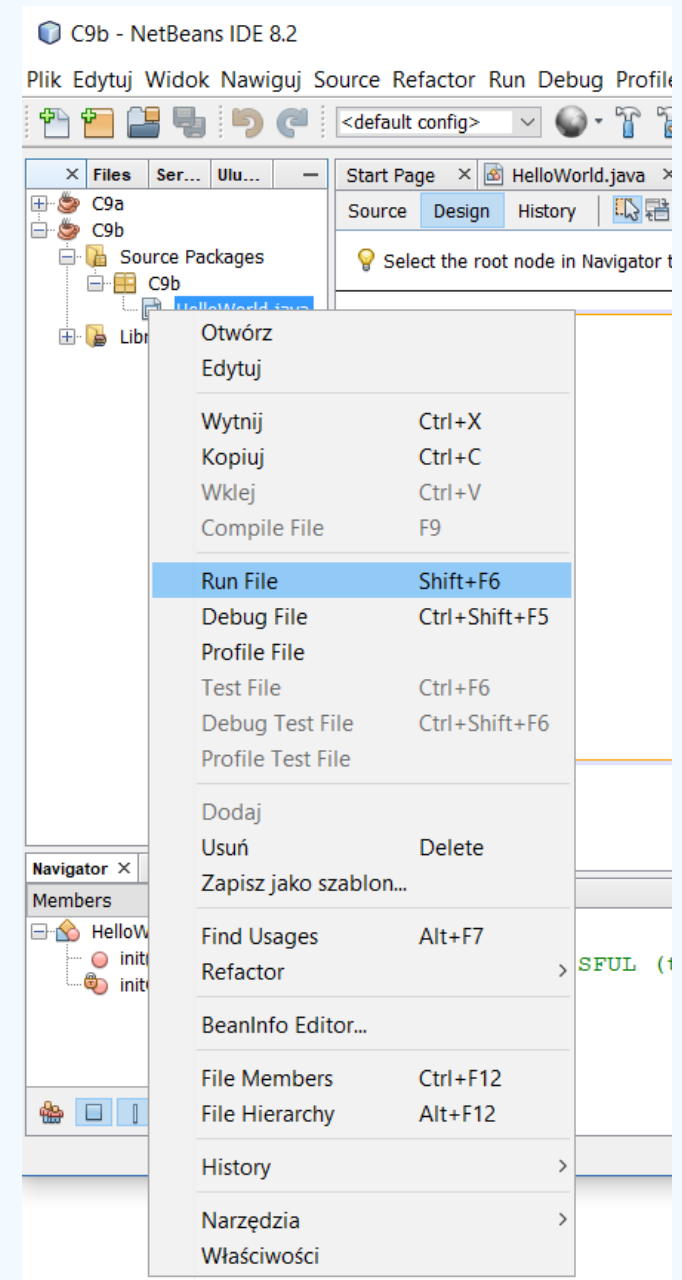
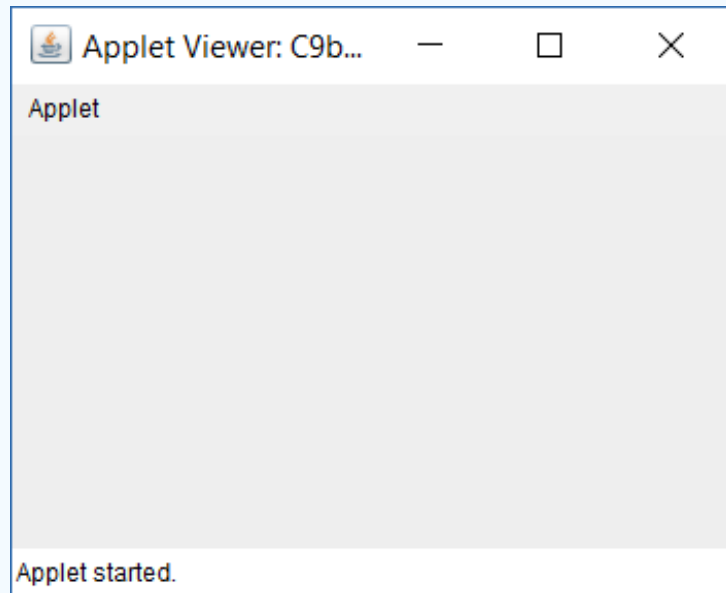
}

@SuppressWarnings("unchecked")
private void initComponents() {
```

```
    javax.swing.GroupLayout layout =  
        new javax.swing.GroupLayout(  
            getContentPane();  
        getContentPane().setLayout(layout);  
        layout.setHorizontalGroup(  
            layout.createParallelGroup(  
                javax.swing.GroupLayout.Alignment.LEADING)  
                .addGap(0, 400, Short.MAX_VALUE)  
            );  
        layout.setVerticalGroup(  
            layout.createParallelGroup(  
                javax.swing.GroupLayout.Alignment.LEADING)  
                .addGap(0, 300, Short.MAX_VALUE)  
            );  
    }  
}
```

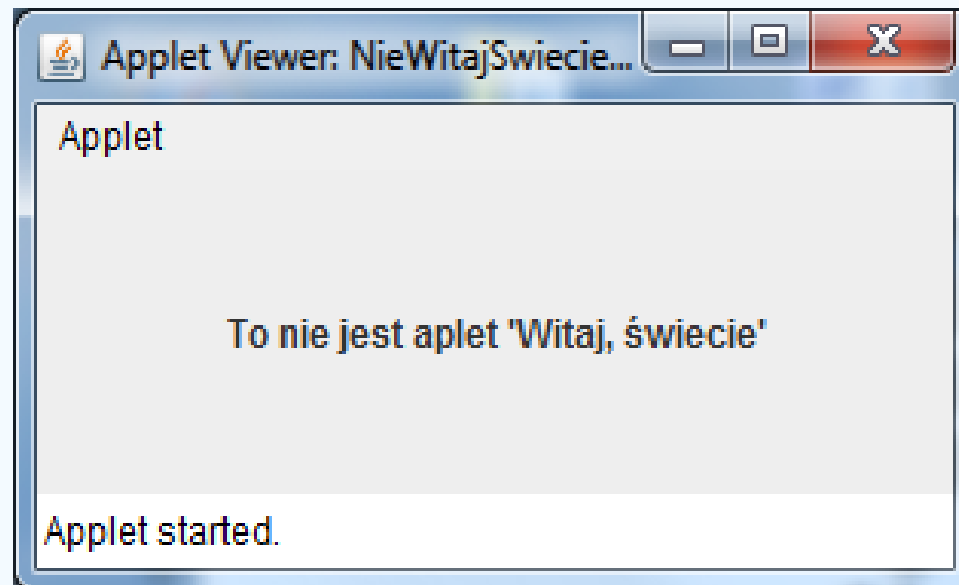
Uruchamianie pliku z apletem w przeglądarce

W celu wyświetlenia apletu, tak jak poprzednio, klikamy prawym klawiszem myszy w ikonę utworzonego pliku i w wyskakującym okienku wybieramy Run File (Shift+F6)



Wyświetlanie apletów za pomocą appletviewer

- Skompilować pliki źródłowe w pliki klas.
- Utworzyć pliki HTML, który poinformuje przeglądarkę, który plik klasy ma załadować, jako pierwszy oraz w jaki sposób ma wyświetlić aplet.
- Wyświetlić plik HTML za pomocą appletviewer np.
C:\G\W9\ApletKalkulatora>appletviewer ApletKalkulatora.html



```
/*  
    <APPLET CODE="NieWitajSwiecieAplet.class" WIDTH=300  
        HEIGHT=100></APPLET>  
*/  
import java.awt.*;  
import javax.swing.*;  
public class NieWitajSwiecieAplet extends JApplet{  
    public void init(){  
        Container powZawartosci = getContentPane();  
        JLabel etykieta = new  
            JLabel("To nie jest aplet 'Witaj, świecie'",  
                SwingConstants.CENTER);  
        powZawartosci.add(etykieta);  
    }  
}
```

Uruchamianie pliku HTML



Architektura apletu

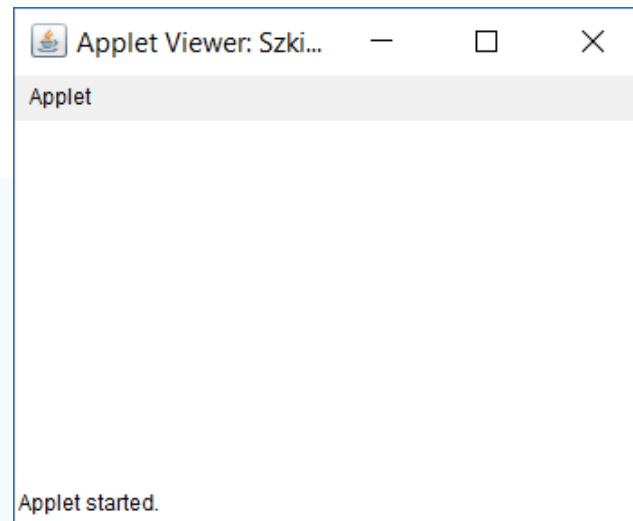
Aplet jest programem opartym na oknach i jako taki ma inną architekturę niż aplikacja konsolowa. Z tego powodu działanie apletów bazuje na obsłudze zdarzeń. Cały proces działania apletu jest realizowany w następujący sposób. Aplet czeka na wystąpienie zdarzenia. Komponent AWT informuje o wystąpieniu zdarzenia przez wywołanie metody obsługującej zdarzenie, która jest przez ten aplet implementowana. Kiedy to nastąpi, aplet podejmuje adekwatne działania i zwraca sterowanie do AWT.

Oprogramowanie apletu polega na przysłonięciu metod, za pośrednictwem których przeglądarki internetowe i przeglądarki apletów kontrolują ich wykonanie. Cztery z nich, *init()*, *start()*, *stop()* i *destroy()*, są definiowane przez klasę *Applet*. Metoda *paint()* jest zdefiniowana przez pakiet AWT klasy *Component*. Szkic apletu został przedstawiony poniżej.

```
import java.applet.Applet;
import java.awt.*;
/*
<applet code="SzkieletApletu" width=300 height=100>
</applet>
*/
public class SzkieletApletu extends Applet {
    // Wywołanie pierwszej metody
    @Override
    public void init() {
        // Inicjalizacja
    }
    // Wywołanie drugiej metody zaraz po metodzie
    // inint(). Metoda start() jest jest wywołana
    // ponownie, gdy aplet jest ponownie uruchamiany
}
```

```
@Override
public void start() {
// wstrzymuje wywołanie
}
// Wywołanie w momencie zatrzymania apletu
@Override
public void stop() {
// Wstrzymuje wywołanie
}
// Wywołanie na końcu działania paletu
@Override
public void destroy() {
// Ostateczne zamknięcie apletu
}
```

```
// Wywołanie w momencie przywrócenia apletu
@Override
public void paint(Graphics g) {
// Ponownie wyświetla zawrtoś apletu
}
}
```



Uwaga: Dostępna jest jeszcze jedna metoda `update()`. Metoda ta jest wywoływana w momencie, w którym aplet musi wyświetlić część swojego okna.

Podstawowe metody wyświetlania składników apletów

Metoda *drawString()* jest wywoływana z poziomu metody *paint()* lub *update()* do wypisania tekstu i jest postaci:

void drawString(**String** *message*, **int** *x*, **int** *y*)

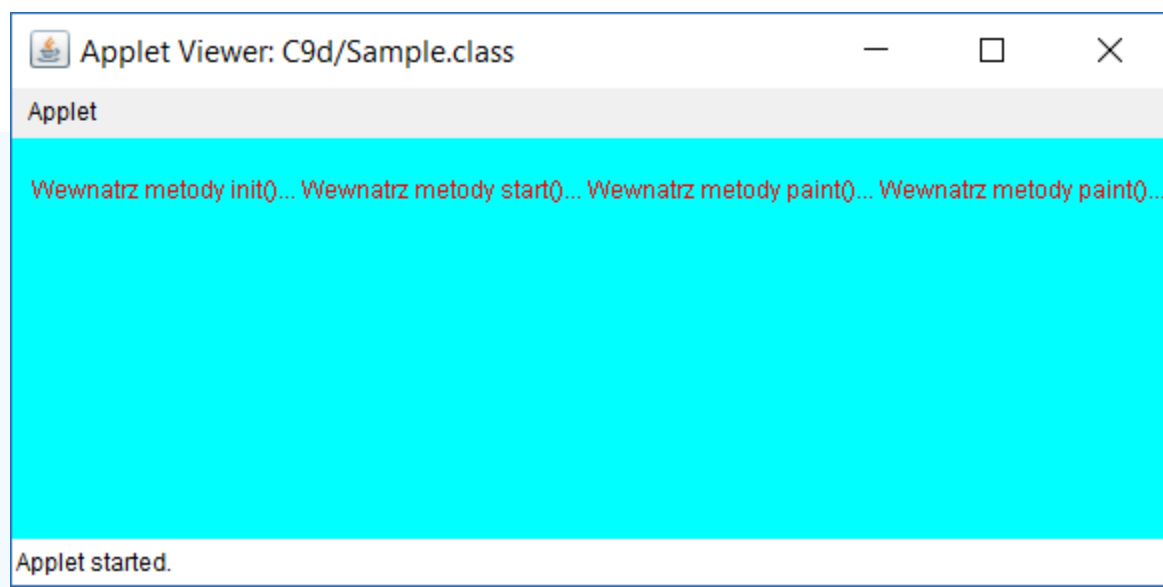
Do ustawienia koloru tła w oknie apletu używamy metody *setBackground()* i do ustawienia koloru pierwszego planu (koloru wyświetlanego tekstu) używamy metody *setForeground()*. Metody są zdefiniowane w klasie *Component*. Ich postać jest następująca:

void setBackground(**Color** *newColor*)

void setForeground(**Color** *newColor*)

```
package C9d;
import java.applet.*;
import java.awt.*;
/**
 * <applet code="Sample" width=300 height=50>
 * </applet>
 */
public class Sample extends Applet {
    String msg;
    @Override
    public void init() {
        setBackground(Color.cyan);
        setForeground(Color.red);
        msg = "Wewnatrz metody init()...";
    }
}
```

```
@Override
public void start() {
    msg += " Wewnatrz metody start()...";
}
@Override
public void paint(Graphics g) {
    msg += " Wewnatrz metody paint()...";
    g.drawString(msg, 10, 30);
}
}
```



Przekazywanie parametrów do apletu

W kodzie HTML znaczniki APPLET umożliwiają przekazywanie parametrów do apletu. Do odczytania wartości parametru służy metoda *getParameter()*. Metoda ta zwraca wartość określonego atrybutu w postaci obiektu klasy String. Oznacza to, że wartości liczbowe i logiczne wymagają przekonwertowania reprezentacji łańcuchów do postaci wewnętrznych formatów. Poniżej przedstawiono przykład mechanizmu przekazywania parametrów:

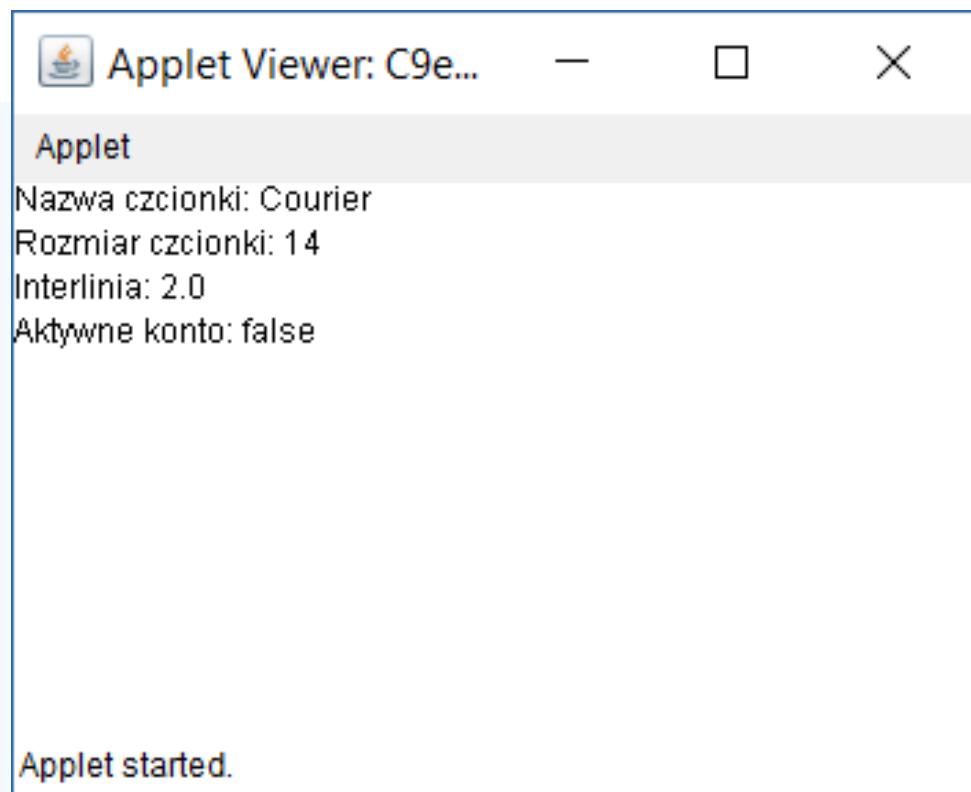
```
package C9e;
import java.applet.*;
import java.awt.*;
/*
<applet code="ParDemo" width=300 height=80>
<parm name=fontName value=Courier>
<parm name=fontSize value=14>
<parm name=leading vale=2>
<parm name=accountEnabled value=true>
  </applet>
  */
public class ParDemo extends Applet {
    String fontName;
    int fontSize;
    float leading;
```

```
boolean active;  
@Override  
public void start() {  
    String parm;  
    fontName = getParameter("fontName");  
    if(fontName == null)  
        fontName = "Courier";  
    parm = getParameter("fontSize");  
    try{  
        if(parm != null)  
            fontSize = Integer.parseInt(parm);  
        else fontSize = 14;  
    } catch(NumberFormatException e) {  
        fontSize = -1;  
    }  
}
```

```
parm = getParameter("leading");
try{
    if(parm != null)
        leading = Float.valueOf(parm);
    else leading = 2.0f;
} catch(NumberFormatException e){
    leading = -1f;}
parm = getParameter("accountEnable");
try{
    if(parm != null)
        active = Boolean.valueOf(parm);
    else active = false;
} catch(Exception e){
    active = false;}
}
```

```
@Override
```

```
public void paint(Graphics g) {  
    g.drawString("Nazwa czcionki: "+fontName, 0, 10);  
    g.drawString("Rozmiar czcionki: "+fontSize, 0, 26);  
    g.drawString("Interlinia: "+leading, 0, 42);  
    g.drawString("Aktywne konto: "+active, 0, 58);  
}  
}
```

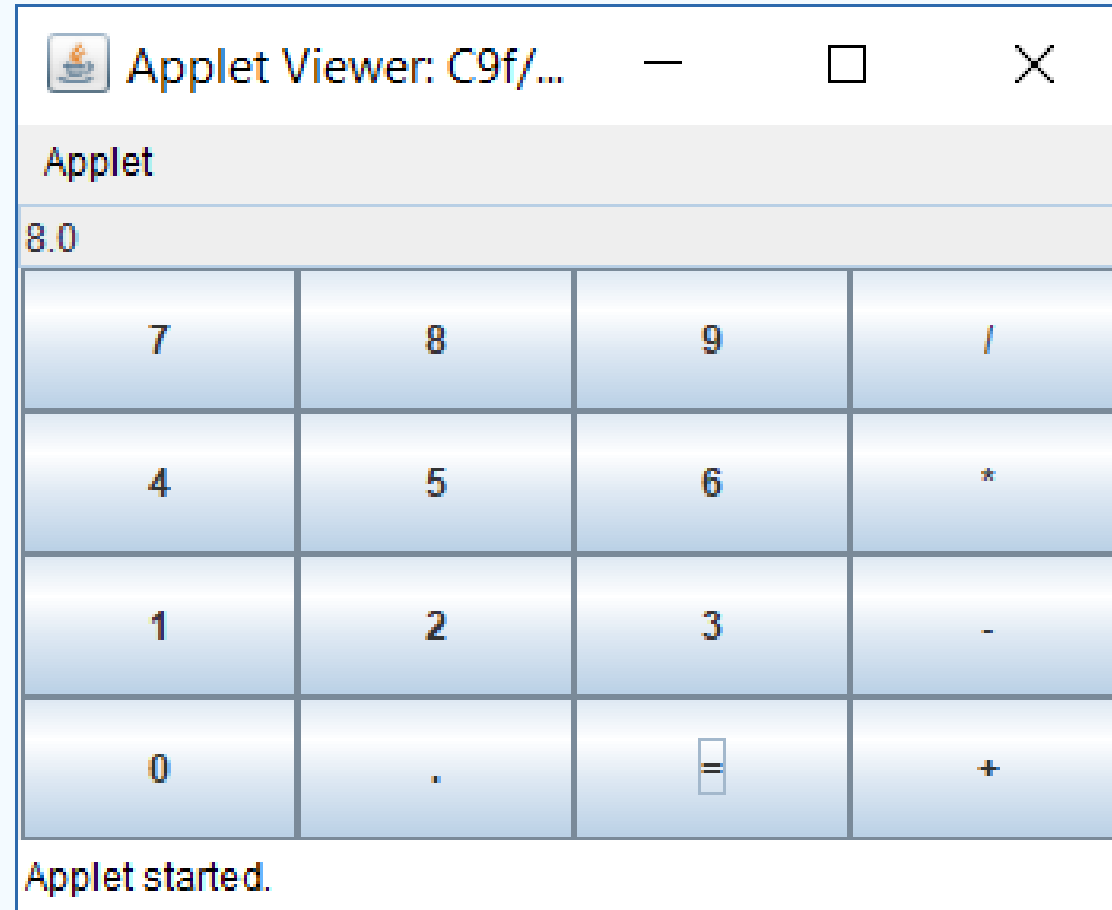


Interfejs AudioClip i AppletStub

Interfejs *audioCip* definiuje następujące metody: *play()* (odtwarza klip audio od początku), *stop()* (zatrzymuje odtwarzanie audio) oraz *loop()* (wymusza ciągłe odtwarzanie klipu). Z powyższy metod możemy korzystać po wczytaniu klipu audio za pomocą metody *getAudioClip()*.

Interfejs *AppletStub* zapewnia mechanizm, za pośrednictwem którego aplet może się komunikować z przeglądarką internetową lub przeglądarką apletów.

Aplet kalkulator



```
package C9f;  
import javax.swing.*;  
import java.awt.*;  
public class ApletKalkulatora  
    extends JApplet {  
    public void init() {  
        Container powZawartosci =  
            getContentPane();  
        PanelKalkulatora panel =  
            new PanelKalkulatora();  
        powZawartosci.add(panel);  
    }  
}
```

```
package C9f;  
import java.awt.*;  
import java.awt.event.*;  
import javax.swing.*;  
class PanelKalkulatora extends JPanel{  
    private JTextField poleWyniku;  
    private JPanel panel;  
    private double wynik;  
    private String ostPolecenie;  
    private boolean poczatek;  
    public PanelKalkulatora()  
    {  
        setLayout (new BorderLayout ());  
        wynik = 0;  
        ostPolecenie = "=";  
        poczatek = true;  
    }
```

```
// dołącz pole wyniku
    poleWyniku = new JTextField("0");
    poleWyniku.setEditable(false);
    add(poleWyniku, BorderLayout.NORTH);
    ActionListener wstaw = new Dolaczenie();
    ActionListener polecenie = new Polecenie();
// dołącz przyciski, ułożone w siatce 4 na 4
    panel = new JPanel();
    panel.setLayout(new GridLayout(4, 4));
    dodajPrzycisk("7", wstaw);
    dodajPrzycisk("8", wstaw);
    dodajPrzycisk("9", wstaw);
    dodajPrzycisk("/", polecenie);
    dodajPrzycisk("4", wstaw);
    dodajPrzycisk("5", wstaw);
```

```
dodajPrzycisk("6", wstaw);
dodajPrzycisk("*", polecenie);
dodajPrzycisk("1", wstaw);
dodajPrzycisk("2", wstaw);
dodajPrzycisk("3", wstaw);
dodajPrzycisk("-", polecenie);
dodajPrzycisk("0", wstaw);
dodajPrzycisk(".", wstaw);
dodajPrzycisk("=", polecenie);
dodajPrzycisk("+", polecenie);
add(panel, BorderLayout.CENTER);
}
// Dołącz przycisk do środkowego panelu.
// @param etykieta etykieta przycisku
// @param sluchacz słuchacz przycisku
```

```
private void dodajPrzycisk(String etykieta,  
                           ActionListener sluchacz){  
    JButton przycisk = new JButton(etykieta);  
    przycisk.addActionListener(sluchacz);  
    panel.add(przycisk);  
}  
//Poniższe działanie powoduje dołączenie  
// stringu klikniętego przycisku na końcu  
// tekstu, znajdującego się w polu wyniku.  
private class Dolaczenie implements  
        ActionListener{  
    @Override  
    public void actionPerformed  
        (ActionEvent zdarzenie){  
String wejscie = zdarzenie.getActionCommand();
```

```
        if (poczatek) {
            poleWyniku.setText("");
            poczatek = false;
        }
        poleWyniku.setText
            (poleWyniku.getText() + wejscie);
    }
}

// Poniższe działanie powoduje wywołanie polecenia,
// opisywanego przez kliknięty przycisk.
private class Polecenie
    implements ActionListener{
    @Override
    public void actionPerformed(ActionEvent zd) {
        String polecenie = zd.getActionCommand();
```

```
String polecenie = zd.getActionCommand();  
if (poczatek){  
    if (polecenie.equals("-")){  
        poleWyniku.setText(polecenie);  
        poczatek = false;  
    }  
    else  
        ostPolecenie = polecenie;  
}  
else{  
    oblicz(Double.parseDouble  
           (poleWyniku.getText()));  
    ostPolecenie = polecenie;  
    poczatek = true;  
}  
}}
```

```
// Przeprowadza wymagane obliczenia.  
// @param x wartość, która wraz z poprzednim  
// wynikiem, po wykonaniu określonego obliczenia,  
// obliczenia, ma dać nowy wynik.  
public void oblicz(double x) {  
    if (ostPolecenie.equals("+")) wynik += x;  
    else if (ostPolecenie.equals("-")) wynik -= x;  
    else if (ostPolecenie.equals("*")) wynik *= x;  
    else if (ostPolecenie.equals("/")) wynik /= x;  
    else if (ostPolecenie.equals("=")) wynik = x;  
    poleWyniku.setText("" + wynik);  
}  
}
```

